Name : Suravi Pubudu Kumara
Mobile : 0478715750
Email : suravipk@gmail.com

Git Repository: https://github.com/suravi999/WeatherMonitor.git

Weather Monitor Solution Overview

This Weather Monitor system demonstrates Clean Architecture principles with a clear separation of concerns across four distinct layers. The solution consists of a Domain layer (named as Core project) containing core business entities (WeatherObservation, WeatherStation), an Application layer implementing business logic and use cases (WeatherService with average temperature calculation algorithms), an Infrastructure layer handling external integrations (Bureau of Meteorology API intergration), and Presentation layers including both a Console UI and RESTful Web API.

The system retrieves real-time weather observation data from the Australian Bureau of Meteorology's JSON feeds and calculates 72-hour temperature averages using precise UTC time calculations between the current UTC time and the observation UTC timestamps. The solution provides flexible data access through both GET endpoints for simple queries and POST endpoints for specific field selection. The Repository Pattern ensures data access abstraction, while dependency injection enables testability and maintainability.

All responses include the required default fields (station name, WMO ID, average temperature) while supporting additional data requests through JSON body parameters. The architecture follows SOLID principles, making it easily extensible for future enhancements while maintaining high code quality and professional enterprise standards suitable for the Business Information Systems team's requirements.

The codebase maintains a medium level of code commenting for complex business logic while emphasizing self-documenting code through descriptive class names, meaningful method names, and well-defined properties and variables. This approach ensures code readability and maintainability without over-commenting, following industry best practices for enterprise software development.The WebAPI includes integrated Swagger documentation for comprehensive API exploration, testing, and developer onboarding.

Due to the limited timeframe, comprehensive unit tests were implemented specifically for the WebAPI project, providing thorough coverage of all controller endpoints and business scenarios. The solution fully satisfies all specified UI and WebAPI requirements as documented in the accompanying technical specification.

UI Requirements Implementation

1. Default to Adelaide Airport with flexible station selection.

- Console app defaults to Adelaide Airport (94672) when run without arguments.
- Accepts both WMO ID (94672) and station names ("Adelaide Airport") as input.
- Station search via API call to find stations by name or WMO ID.

2. Calculate and present 72-hour average temperature.

- By default 72-hour temperature averaging with configurable time range.
- Only includes observations with valid temperature data.
- Returns both average temperature and count of observations used.

3. Always display required default fields.

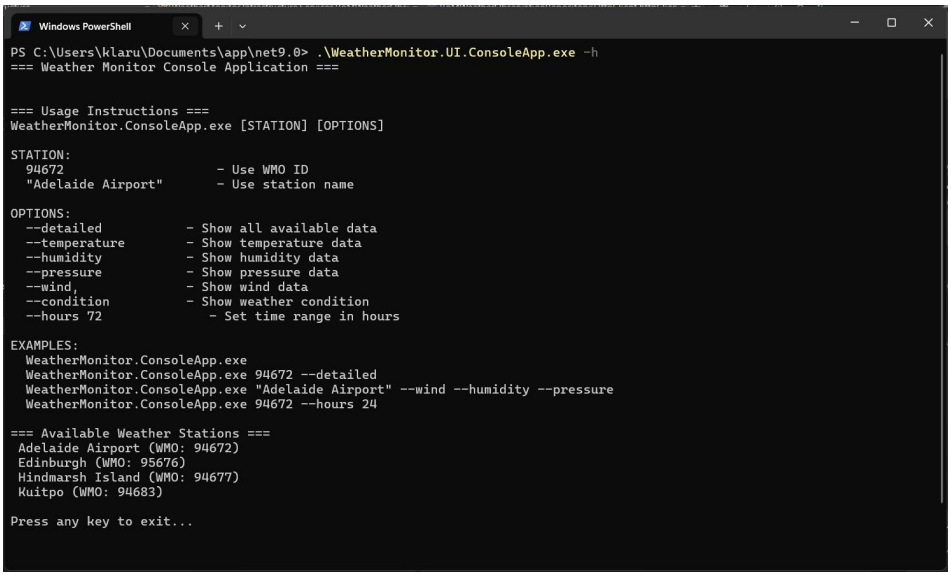
- All Summary API endpoints return station name, WMO ID, and average temperature.
- Console app always displays these three required fields first.
- User can request the additional fileds via console arguments.
- Additional requested data is shown below the required fields.

4. User-friendly station identification (no WMO knowledge required).

- Implemented search and lookup functionality
  - Station search by partial name: "Adelaide", "airport", "island".
- Case insensitive station name matching.
- Console app accepts station names

5. Help Feature

- Print all instruction and Lists all available stations for user reference with the help command line argument.
  >WeatherMonitor.exe --help



API Requirements Implementation

1. Retrieve all weather observation data for any weather observation station.

```
curl -X 'GET' \
  'https://localhost:44303/api/WeatherObservation/95676' \
  -H 'accept: */*'
```

Or

```
curl -X 'POST' \
  'https://localhost:44303/api/WeatherObservation/95676' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "requestedDataTypes": [ ]
}'
```

2. Retrieve a specific piece of weather observation data for any weather observation station.

```
curl -X 'POST' \
  'https://localhost:44303/api/WeatherObservation/95676' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "requestedDataTypes": [
    "dewPoint"
  ]
}'
```

3. All API responses use JSON format

4. A RESTful HTTP service, deployable to an IIS web server.